

Project Based Learning Report on
DATA CLEANING AND VISUALIZATION

In partial fulfilment of the requirement

For Project Based Learning of

BACHELOR OF TECHNOLOGY

In

COMPUTER ENGINEERING

For

Data Warehousing & Mining



BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

COLLEGE OF ENGINEERING, PUNE-43

DEPARTMENT OF COMPUTER ENGINEERING

2026-27

SUBMITTED TO:

Dr. Rohini Khalkar

SUBMITTED BY:

PRN

Ankit Akash(1)

2414110181

Shivam Gupta(25)

2414110209

Prashant Kumar (41)

2414110233

Krishna Mishra (50)

2414110248

**BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)
COLLEGE OF ENGINEERING, PUNE-43 DEPARTMENT
OF COMPUTER ENGINEERING**



CERTIFICATE

This is to certify that the Project Based Learning report titled “**DATA CLEANING AND VISUALIZATION**”, submitted by Ankit Akash (1), Shivam Gupta (25), Prashant Kumar (41), Krishna Mishra (50), to the Bharati Vidyapeeth (Deemed to be University), College of Engineering, Pune-43 for the award of the degree of **BACHELOR OF TECHNOLOGY** in Computer Engineering is a Bonafide record of the PBL work done by them under my supervision.

Place: Pune
Date:

Dr. Rohini Khalkar

Acknowledgement

To begin with, we would like to thank our Course Coordinator for providing us this golden opportunity. We are very grateful to our Course Coordinator, **Dr. Rohini Khalkar** for her valuable support and guidance.

We are also very grateful to all those who helped us in our entire project. We would like to thank **Dr. Rohini Khalkar**, our Course Coordinator and our **HOD Dr. S.B. Vanjale** sir for their support and guidance in completing our project-based learning on the topic “DATA CLEANING AND VISUALIZATION”. It was a great learning experience.

I would like to take this opportunity to express my gratitude to all the group members Ankit, Shivam, Prashant, Krishna. The project-based learning would not have been successful without their cooperation and inputs. At last, but not the least, we are thankful to our parents and families who encourage us to study and who support us financially and who gave us the opportunity to spend our golden time in learning.

Thanks again to all who helped us.

Index

S.NO	TITLE	PAGE NO.
1	Introduction	1
2	Objectives	2
3	System Requirements	3
4	Architecture	4
5	Implementation	5
6	Code	7
7	Output	9
8	Advantage & Disadvantage	10
9	Accuracy & Efficiency	11
10	Challenges	12
11	Future Scope and Enhancements	13
12	Conclusion	14
13	References	15

1. Introduction

In modern e-commerce environments, sales data is generated continuously from customer orders, products, quantities, prices, locations, and transaction dates. Converting this raw information into useful business knowledge requires reliable data storage, preprocessing, aggregation, and visualization. The Amazon Sales Pulse project addresses this requirement by providing a compact analytics dashboard that combines a data pipeline with customer and sales analysis.

The application is designed as a local web dashboard. A Flask backend exposes analytical API endpoints, while the front end uses HTML, CSS, JavaScript, and Chart.js to present interactive charts and tables. The Excel workbook `sales_data.xlsx` acts as the project data store. The pipeline can operate in a demonstration mode that generates realistic synthetic orders or can be configured for an Amazon Selling Partner API connection using the seller's own credentials.

The project follows a simple Extract–Transform–Load concept. Data is obtained from a source, transformed into a common order structure, duplicate OrderIDs are removed during loading, and the cleaned records are appended to Excel. The dashboard then reads the workbook and calculates revenue, orders, units, customer segments, repeat rate, lifetime value, product performance, and city-wise revenue.

The supplied project README explicitly describes the application as a creative local dashboard for analysing customers and sales, with data saved to Excel. It also notes that real Amazon customer data is not publicly available and that the default source is a simulated demo feed. This distinction is important for ethical and technically accurate project presentation.

Problem Statement

Small and medium-scale analytics demonstrations often require a single interface through which raw sales records can be stored, monitored, and converted into meaningful business indicators. A system is therefore required that can maintain order records, support manual entry, demonstrate a data pipeline, calculate sales trends at multiple time granularities, analyse customer behaviour, and present the results through an understandable dashboard.

Scope of the Project

- Sales and revenue analysis using order-level data.
- Customer segmentation based on order frequency.
- Daily, weekly, monthly, and yearly sales views.

- Manual insertion of new order records into Excel.
- Pipeline monitoring with demo or Amazon SP-API source.
- Download/export of the Excel sales dataset.

Interactive visualization through charts, tables, KPI cards, and a pipeline status log

2. Objectives

1. Develop an interactive web-based dashboard for sales and customer analytics.
2. Create a structured order dataset containing customer, product, category, quantity, price, date, and source fields.
3. Implement a data pipeline following Extract–Transform–Load principles.
4. Store order data persistently in an Excel workbook using OpenPyXL.
5. Provide daily, weekly, monthly, and yearly sales analysis.
6. Analyse customer behaviour using order frequency, repeat rate, and lifetime value.
7. Allow new records to be entered through a web form and saved directly to Excel.
8. Demonstrate a live-feed concept through automatic pipeline execution.
9. Provide an optional integration path for Amazon Selling Partner API data.
10. Present analytical results in a visually appealing and easy-to-understand interface.

Learning Outcomes

- Understanding the relationship between data ingestion, storage, preprocessing, and visualization.
- Practical use of Python, Flask, OpenPyXL, HTML, CSS, and JavaScript.
- Design of REST-style endpoints for analytics data.
- Application of aggregation logic for business intelligence.
- Understanding the limitations of demo data versus real marketplace data.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or higher; Intel i5/i7 recommended for smoother development.
- RAM: Minimum 4 GB; 8 GB recommended.
- Storage: At least 500 MB free space for Python environment, project files, and generated Excel data.
- Display: 1366 × 768 or higher recommended for comfortable dashboard viewing.
- Internet connection: Required initially for browser-loaded Chart.js/fonts and optional Amazon API access.

Software Requirements

Component	Requirement	Purpose
Operating System	Windows 10/11, Linux, or macOS	Run the local application
Python	3.9+ recommended	Backend and pipeline implementation
Flask	3.x or compatible	Web server and API endpoints
OpenPyXL	3.1+	Excel read/write operations
Requests	2.31+	HTTP requests for SP-API
python-dotenv	1.0+	Environment configuration
Browser	Chrome/Edge/Firefox	Dashboard interface
Excel	Microsoft Excel or compatible	Open/export sales workbook

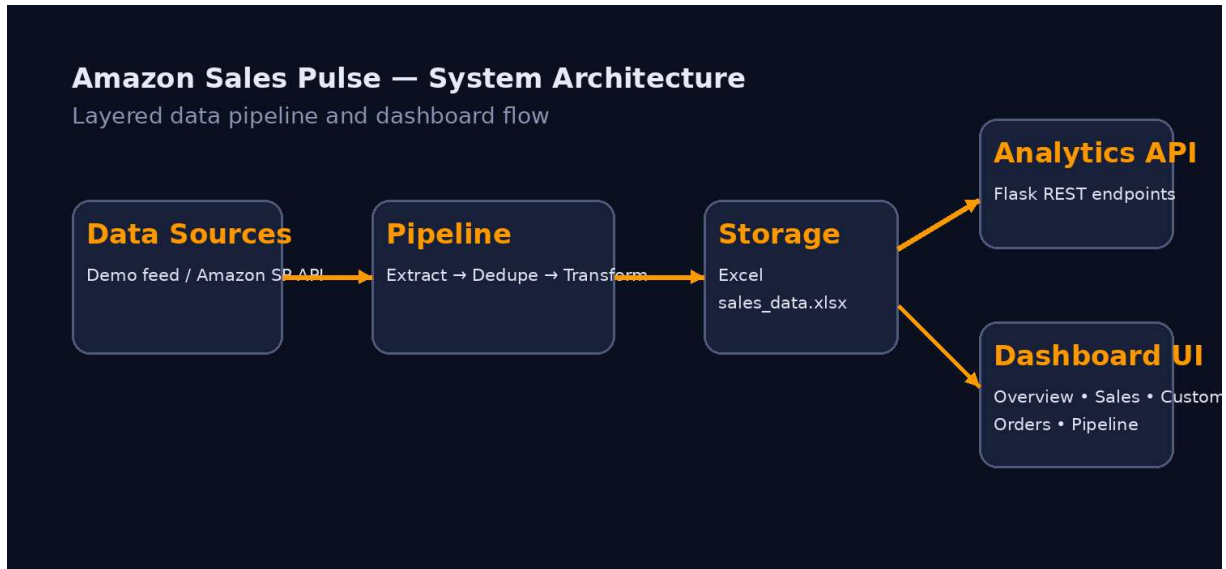
Project File Structure

```
amazon_dashboard/
├── app.py           # Flask application and analytics APIs
├── pipeline.py      # Demo/SP-API source and ETL logic
├── storage.py       # Excel storage layer
├── requirements.txt  # Python dependencies
├── run.bat / run.sh # Startup scripts
├── .env.example     # API configuration template
└──
```

Bharati Vidyapeeth (Deemed to be University) College of Engineering, Pune

- |— templates/index.html # Dashboard interface
- |— static/
 - |— app.js # Front-end logic and charts
 - |— style.css # Dashboard styling

4. ARCHITECTURE



The architecture is divided into logical layers. The source layer provides either generated demonstration orders or an optional Amazon Selling Partner API source. The pipeline layer performs extraction and loading with duplicate prevention. The storage layer maintains records in sales_data.xlsx. The Flask analytics layer reads the stored records and exposes JSON endpoints. Finally, the dashboard layer consumes these endpoints and displays KPIs, charts, tables, forms, and pipeline status.

Data Flow

11. Source data is generated by DemoSource or retrieved from the configured SP-API source.
12. Each record is converted to a common order representation containing OrderID, Date, CustomerID, CustomerName, Email, City, Product, Category, Qty, UnitPrice, and Source.
13. storage.append_orders() checks existing OrderIDs and skips duplicates.

14. The Total field is calculated from $\text{quantity} \times \text{unit price}$ before the record is written to Excel.
15. Flask API endpoints read the Excel data and aggregate it according to the requested analysis.
16. The JavaScript front end requests JSON data and renders charts and tables with Chart.js.

5. Implementation Details

5.1 Application Startup

The application starts through app.py. During bootstrap, the program checks whether the Excel workbook exists. If it does not exist, pipeline.generate_history() creates approximately 900 days of demonstration orders and storage.init() creates the workbook and Orders sheet. The Flask server then exposes the dashboard and API endpoints on the local machine.

```
def bootstrap():  
    if not os.path.exists(storage.XLSX_PATH):  
        print("First run: creating sales_data.xlsx with sample history...")  
        storage.init(pipeline.generate_history())
```

5.2 Data Generation

The demo source is intentionally deterministic for the initial history. It maintains a customer pool of 1,500 possible synthetic customers, uses a product catalogue spanning Electronics, Home, Fashion, Sports, Books, Grocery, and Beauty, and varies order volume using growth, weekend, and festive-season factors. This produces non-empty charts suitable for demonstration.

5.3 Excel Storage

The storage layer creates an Orders worksheet with twelve columns. The workbook is styled with a header row, column widths, date formatting, and frozen panes. A threading lock is used to protect workbook access inside the local application. New entries are appended while duplicate OrderIDs are ignored.

Field	Description
OrderID	Unique order identifier
Date	Order date
CustomerID	Customer key
CustomerName	Customer display name
Email	Customer email or identifier
City	Customer city

Product	Purchased product
Category	Product category
Qty	Quantity ordered
UnitPrice	Unit price in ₹
Total	Calculated order value
Source	demo, manual, or amazon-spapi

5.4 Analytics Modules

- Overview API: calculates total revenue, order count, customer count, average order value, today's revenue, recent revenue, category totals, and top products.
- Sales API: groups orders by daily, weekly, monthly, or yearly periods and calculates revenue, order count, units, category mix, best period, and change from the previous period.
- Customers API: groups records by CustomerID and calculates order count, spending, first/last order date, segments, city revenue, new customers, repeat rate, and average lifetime value.
- Orders API: lists recent orders and supports search; POST creates a manual order and saves it to Excel.
- Pipeline API: runs the source-to-Excel process, reports status, enables live mode, and maintains a short execution log.

6.Code

6.1 Flask Analytics Endpoint

```
@app.route("/api/overview")
def overview():
    orders = storage.read_orders()
    rev = sum(o["Total"] for o in orders)
    custs = {o["CustomerID"] for o in orders}
    return jsonify({
        "revenue": round(rev, 2),
        "orders": len(orders),
        "customers": len(custs),
        "aov": round(rev / len(orders), 2) if orders else 0
    })
```

6.2 Sales Period Grouping

```
def pkey(d, p):
    if p == "daily": return d
    if p == "weekly": return d - timedelta(days=d.weekday())
    if p == "monthly": return date(d.year, d.month, 1)
    return date(d.year, 1, 1)
```

6.3 Customer Segmentation

```
seg = {"One-time": 0, "Repeat (2-3)": 0, "Loyal (4+)": 0}
for c in cl:
    if c["orders"] == 1:
        seg["One-time"] += 1
    elif c["orders"] <= 3:
        seg["Repeat (2-3)"] += 1
    else:
        seg["Loyal (4+)"] += 1
```

6.4 Duplicate-Safe Excel Loading

```

existing = {r[0] for r in ws.iter_rows(
    min_row=2, max_col=1, values_only=True) if r[0]}
for o in new_orders:
    if o["OrderID"] in existing:
        continue
    ws.append(_row_to_values(o))
    existing.add(o["OrderID"])

```

6.5 Front-End Visualization

The JavaScript layer uses `fetch()` to request JSON from Flask endpoints and `Chart.js` to render doughnut, bar, and line/area visualizations. Existing charts are destroyed before redraw so that period changes do not create duplicate chart instances. The interface also supports animated KPI values, search, form submission, pipeline controls, and a live-refresh concept.

6. Output

A functional test was performed using the project's built-in deterministic demonstration generator. The resulting Excel workbook was read through the same storage layer used by the application. The following values therefore describe the supplied demo dataset after initialization, not real Amazon seller activity.

Metric	Observed demo result
Orders	3,393
Distinct customers	1,218
Units sold	4,860
Total revenue	₹1,38,75,708.22
Average order value	₹4,089.51
Product categories	7
History generated	Approximately 900 days
Top revenue category	Electronics – 58.16% of revenue
Repeat customers	67.32% of customers
Loyal customers (4+ orders)	294

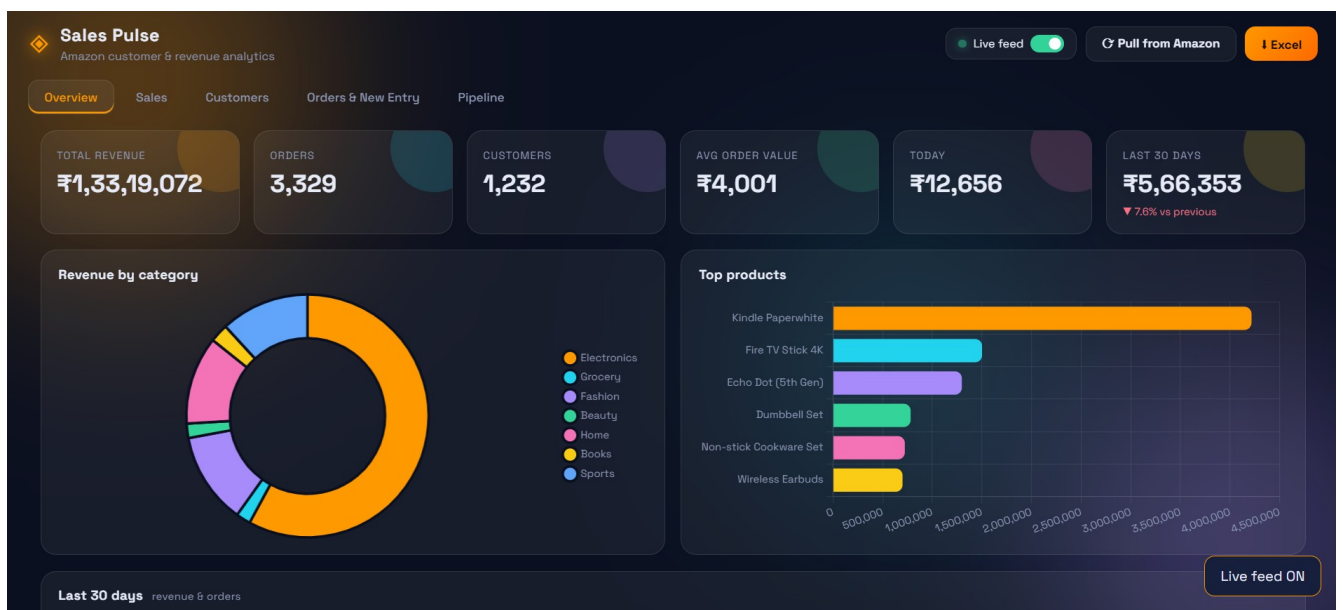
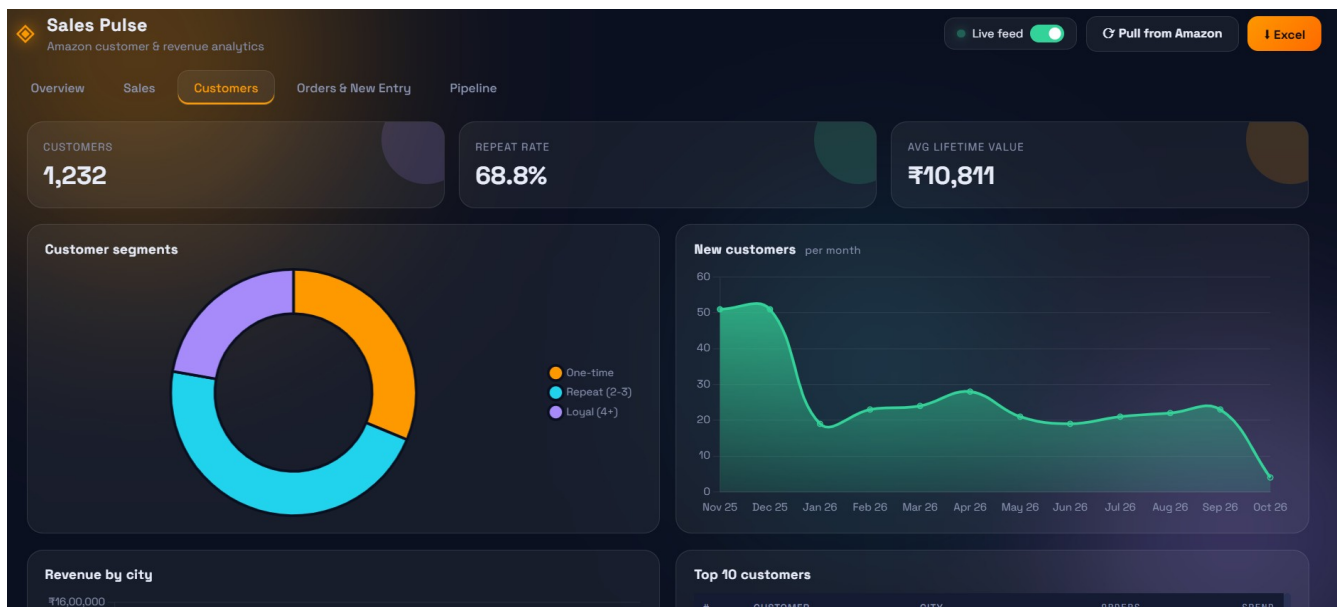
Category Analysis

Category	Revenue (₹)	Share
Electronics	80,69,843.32	58.16%
Sports	16,84,146.04	12.14%
Home	16,47,563.10	11.87%
Fashion	16,16,353.79	11.65%
Books	3,14,905.11	2.27%
Grocery	2,71,806.04	1.96%
Beauty	2,71,090.82	1.95%

Top Products by Revenue

Product	Revenue (₹)
Kindle Paperwhite	45,32,953.99
Fire TV Stick 4K	15,39,223.65
Echo Dot (5th Gen)	13,23,490.98
Dumbbell Set	8,70,011.30
Non-stick Cookware Set	7,36,789.48
Wireless Earbuds	6,74,174.70





8.Advantages & Disadvantages

Advantages

- Simple architecture suitable for a mini-project and academic demonstration.
- Interactive dashboard provides multiple analytical views in one interface.
- Excel storage is easy to inspect, share, and understand for beginners.
- Manual entry allows immediate testing of how a new record changes analytics.
- Daily, weekly, monthly, and yearly analysis supports multiple reporting requirements.
- Customer segmentation and repeat-rate analysis provide basic customer intelligence.
- The pipeline separates data acquisition from storage and visualization.
- Optional SP-API integration provides a path toward real seller data without scraping public Amazon pages.

Disadvantages / Limitations

- Excel is not ideal as a high-concurrency production database.
- The default dataset is synthetic and therefore does not represent real Amazon sales.
- Live SP-API operation requires the user's own Amazon seller credentials and appropriate permissions.
- The dashboard depends on the local machine for the Flask server and workbook.
- Advanced predictive analytics and machine-learning models are not included in the current version.
- The project does not implement enterprise authentication, role-based access, or multi-user database transactions.

9. Accuracy & Efficiency

Because this project is primarily an analytics dashboard rather than a predictive machine-learning model, conventional classification accuracy is not the appropriate evaluation metric. Instead, correctness was assessed through functional and data-consistency checks.

9.1 Functional Accuracy Checks

- The Total value for each order is derived from $\text{Qty} \times \text{UnitPrice}$ before storage.
- Customer count is calculated from distinct CustomerID values.
- Revenue totals are obtained by summing stored order totals.
- Sales periods are grouped using explicit daily, weekly, monthly, and yearly keys.
- Duplicate OrderIDs are skipped during pipeline loading.
- Customer repeat rate is calculated from customers having more than one order.

9.2 Efficiency Considerations

The application reads the workbook for each analytics request, which is practical for a small academic dataset but becomes less efficient as the number of records increases. The use of a lock helps prevent conflicting workbook access inside the local application. The project would become substantially more scalable by replacing Excel with a relational database and by caching or pre-aggregating frequently requested metrics.

9.3 Demonstrated Dataset

Check	Result
Generated order records	3,393
Distinct CustomerID values	1,218
Calculated units	4,860
Calculated total revenue	₹1,38,75,708.22
Duplicate handling	Implemented by OrderID
Time granularities	4

10. Challenges

During development, several practical challenges were addressed. The first was designing a dataset rich enough to make the dashboard meaningful without relying on private or inaccessible customer information. The solution was a controlled demo generator with synthetic customers, products, cities, seasonal variation, and growth.

A second challenge was maintaining a simple storage mechanism while still supporting repeated updates. OpenPyXL was used to create, read, and append workbook records, and a threading lock was added around workbook operations. Duplicate OrderIDs are checked before append operations so that repeated pipeline runs do not automatically create duplicate records.

Another challenge involved the difference between a simulated live feed and real Amazon data. The project therefore keeps the demo source as the default and includes a separate SPAPISource implementation. The README explicitly notes that real customer data is subject to Amazon permissions and that the SP-API connector may require field mapping adjustments for a particular seller account.

The final challenge was presenting multiple analytical dimensions without making the dashboard difficult to use. Tabs, KPI cards, responsive grids, charts, searchable tables, and a dedicated pipeline view were used to separate concerns while keeping the application visually consistent.

11.Future Scope and Enhancements

17. Replace Excel with MySQL, PostgreSQL, or another transactional database for scalability.
18. Add user authentication and role-based authorization for administrators and analysts.
19. Implement scheduled background jobs using a production task scheduler.
20. Add forecasting models for future sales and demand prediction.
21. Introduce product recommendation algorithms based on customer purchase behaviour.
22. Add advanced customer segmentation using RFM analysis or clustering.
23. Provide downloadable PDF reports and scheduled email summaries.
24. Add filters for category, city, product, date range, and customer segment.
25. Improve SP-API integration with robust pagination, rate-limit handling, error recovery, and account-specific field mapping.
26. Add automated tests for storage, analytics endpoints, pipeline deduplication, and input validation.
27. Deploy the dashboard to a secure cloud environment with a managed database.

12.Conclusion

Amazon Sales Pulse – Customer & Sales Analytics Dashboard demonstrates how data warehousing and mining concepts can be applied to an e-commerce analytics scenario using a lightweight web application. The project combines data generation or ingestion, transformation, persistent storage, analytical aggregation, and interactive visualization into one workflow.

The implementation successfully demonstrates sales reporting at daily, weekly, monthly, and yearly levels, customer segmentation, repeat-rate analysis, lifetime-value calculation, city-wise revenue analysis, product ranking, manual data entry, Excel export, and pipeline monitoring. The generated demonstration dataset provides enough history to make these analytical views meaningful during a project presentation.

The project also demonstrates an important practical distinction between simulated data and real marketplace information. The default demo mode is suitable for academic testing, while real Amazon data requires an authorized seller account and Amazon Selling Partner API credentials. Consequently, the current project is best viewed as an educational analytics platform and a foundation for future production-oriented enhancements.

Overall, the project provides practical experience in Python-based data processing, REST API development, spreadsheet-based storage, front-end visualization, and data-driven decision support. It meets the objectives of a Data Warehousing & Mining mini-project by showing the complete path from data source to analytical insight.

13.References

- Project source files supplied for this PBL: app.py, pipeline.py, storage.py, templates/index.html, static/app.js, static/style.css, README.md.
- Python Documentation — <https://docs.python.org/3/>
- Flask Documentation — <https://flask.palletsprojects.com/>
- OpenPyXL Documentation — <https://openpyxl.readthedocs.io/>
- Chart.js Documentation — <https://www.chartjs.org/docs/>
- Amazon Selling Partner API Documentation — <https://developer-docs.amazon.com/sp-api/>